

techBuilder

Multi-Tenant Construction Management Platform

System Architecture & Design Document

Version 1.0 — MVP Phase
March 2026

Prepared for: Senior Engineering Team & Stakeholders
Classification: Internal — Technical Architecture

Table of Contents

Table of Contents.....	2
1. High-Level System Architecture	5
1.1 System Component Overview	5
1.2 Request Flow Architecture.....	6
1.3 Multi-Tenancy Architecture	6
2. Page-by-Page Breakdown — Complete Application Map.....	8
2.1  Worker (Labourer) — 6 Pages	8
Worker Home.....	8
Worker Photo Upload.....	9
Worker View Task.....	9
Worker Attendance History	10
Worker Profile	10
Worker Leave Request	10
2.2  Driver — 8 Pages.....	11
Driver Home.....	11
Driver Start Day Log	11
Driver End Day Log.....	11
Driver Fuel Entry	12
Driver Expense Entry	12
Driver Vehicle Issue Report	12
Driver Daily Summary	13
Driver Trip Photo Upload.....	13
2.3  Worker-Head (Mistri) — 8 Pages	14
Worker-Head Home	14
Worker-Head Crew Attendance	14
Worker-Head Upload on Behalf	14
Worker-Head Progress Report.....	15
Worker-Head Material Log	15
Worker-Head Issue Report	15
Worker-Head Daily Note	16
Worker-Head Expense Entry.....	16
2.4  Sub-Manager (Site Manager) — 11 Pages	18
Sub-Manager Home / Site Dashboard	18
Sub-Manager Attendance View.....	18

Sub-Manager Daily Photo Upload	18
Sub-Manager Evening Report.....	19
Sub-Manager Task Assignment	19
Sub-Manager Expense Management.....	19
Sub-Manager Progress Tracker	20
Sub-Manager Worker List	20
Sub-Manager Vehicle View	20
Sub-Manager Issue Tracker.....	21
Sub-Manager Material Request	21
2.5 🗑 Manager (Company Owner/Admin) — 14 Pages	22
Manager Dashboard	22
Manager All Sites View	22
Manager Attendance Overview	22
Manager Fleet Overview	23
Manager Activity Feed	23
Manager Expense Dashboard.....	23
Manager Approve Expenses	24
Manager Worker Directory	24
Manager Vehicle Directory.....	24
Manager Reports & Downloads	25
Manager Assign Workers.....	25
Manager Assign Vehicles.....	25
Manager Push Notifications Center.....	26
Manager Site Detail View.....	26
2.6 🛡 Platform (Super Admin) — 5 Pages	27
Super Admin Dashboard.....	27
Tenant Management.....	27
Subscription & Billing	27
Platform Audit Log	28
System Backups	28
2.7 🔗 Shared / Cross-Role Pages — 6 Pages	29
Login / Authentication	29
Company Creation / Onboarding.....	29
GPS Tracking Consent	29
Settings.....	30
Photo Gallery	30

Notification Center	30
3. Data Flow Diagrams.....	32
3.1 Photo Upload Flow (Offline → Sync)	32
3.2 Location Tracking Flow	32
3.3 Report Generation Flow.....	33
3.4 Offline Sync Flow (PowerSync)	34
4. Complexity & Risk Analysis.....	35
4.1 Hardest Parts of the System	35
4.2 Scaling Challenges.....	35
4.3 Security Concerns	36
5. Feature Phases — Development Roadmap	37
5.1 Phase 1 — MVP (Months 1-5)	37
5.2 Phase 2 — Growth (Months 6-10)	37
5.3 Phase 3 — Pro (Months 11-18)	38
6. Initial Budget — What's Free, What's Not.....	39
6.1 Free Tier Stack (₹0/month)	39
6.2 First Paid Services (When Revenue Starts)	39
7. Developer Skill Requirements	41
7.1 Minimum Team for MVP	41
8. Suggestions & Missing Features	43
8.1 Features Commonly Found in Competitors That You Should Add	43
8.2 Competitive Differentiators	43
8.3 Critical Missing Items to Add Before Launch.....	44
9. Week-by-Week Sprint Plan — MVP (20 Weeks)	45
10. Project Structure — Monorepo Layout	47
11. Conclusion & Next Steps.....	48

1. High-Level System Architecture

techBuilder is a multi-tenant construction management SaaS consisting of three primary surfaces: a Next.js web dashboard (for managers/admins), a React Native mobile app (for field workers, drivers, and site managers), and a NestJS backend with PostgreSQL. The system is designed offline-first for mobile, with real-time sync for the web dashboard.

1.1 System Component Overview

The following diagram shows how all major system components connect:

Component	Technology	Purpose & Notes
Web Dashboard	Next.js 16 + Tailwind v4 + shadcn/ui	Admin/Manager desktop experience. Server-rendered for SEO on public pages, client-rendered for dashboard. Hosted on Vercel.
Mobile App	Expo SDK 55 + React Native + NativeWind	Worker/Driver/Site Manager field app. Offline-first via PowerSync + SQLite. Camera, GPS, push notifications. Published to Play Store.
API Backend	NestJS v11 + Fastify + Drizzle ORM	REST API with WebSocket support. Handles auth, business logic, file processing, tenant isolation. Modular monolith architecture.
Database	PostgreSQL 16 + PostGIS + TimescaleDB	Shared-schema multi-tenancy with Row-Level Security. PostGIS for geospatial queries. TimescaleDB for time-series (location tracking, analytics).
File Storage	Cloudflare R2 (S3-compatible)	Construction photos, receipt images, vehicle documents. Zero egress fees. CDN-backed delivery via Cloudflare.
Offline Sync	PowerSync + SQLite	Bidirectional sync between mobile SQLite and PostgreSQL. Handles conflict resolution. Critical for construction sites with poor connectivity.
Real-Time	Socket.io on NestJS	WebSocket connections for live attendance updates, fleet tracking, and notification delivery on the web dashboard.
Push Notifications	Expo Notifications + FCM	Firebase Cloud Messaging (free, unlimited) for Android. Expo handles token management. Critical for issue alerts.
SMS / OTP	MSG91 (₹0.25/SMS)	Indian OTP service for phone-based authentication. DLT compliant. WhatsApp OTP as primary, SMS as fallback.
Payments	Razorpay Subscriptions	UPI AutoPay for subscription billing. Handles RBI e-mandate compliance automatically.
Hosting	Vercel (web) + Neon (DB) + R2 (files)	Vercel for Next.js, Neon serverless PostgreSQL (free tier: 0.5GB), Cloudflare R2 for storage.

Component	Technology	Purpose & Notes
CI/CD	GitHub Actions + Turborepo	Monorepo with Turborepo. GitHub Actions for lint, test, build, deploy. 2,000 free minutes/month.
Monitoring	PostHog + Sentry	PostHog: 1M free analytics events + 5K session replays. Sentry: 5K free error events for crash tracking.

1.2 Request Flow Architecture

Every request in techBuilder follows a clear path through the system. Understanding this flow is essential for every developer on the team.

1. **USER ACTION:** Worker taps 'Clock In' on mobile app
2. **LOCAL FIRST:** PowerSync writes clock-in record to local SQLite immediately (app shows success instantly)
3. **BACKGROUND SYNC:** When network is available, PowerSync pushes the record to the NestJS API
4. **API PROCESSING:** NestJS receives the request, validates tenant context via JWT, applies business rules
5. **DATABASE WRITE:** Drizzle ORM writes to PostgreSQL with automatic `tenant_id` injection via RLS
6. **REAL-TIME BROADCAST:** Socket.io pushes the attendance update to the Manager's web dashboard
7. **PUSH NOTIFICATION:** If attendance triggers an alert (e.g., low attendance), FCM pushes to Manager's phone
8. **AGGREGATION UPDATE:** Pre-computed rollup tables update (today's attendance count for this site)
9. **DASHBOARD REFRESH:** Manager sees the updated attendance count on their dashboard within 2-5 seconds

1.3 Multi-Tenancy Architecture

techBuilder uses shared-schema multi-tenancy with PostgreSQL Row-Level Security (RLS). This means all tenants share the same database tables, but every table has a `tenant_id` column, and RLS policies ensure Company A can never see Company B's data — even if there's a bug in the application code.

Aspect	Implementation
Tenant Provisioning	When a new company signs up, a new row is inserted into the tenants table. All subsequent data for this company includes this <code>tenant_id</code> .
Request Context	Every API request includes a JWT with the user's <code>tenant_id</code> . NestJS middleware extracts it and calls <code>SET LOCAL app.current_tenant = 'uuid'</code> before any query.

Aspect	Implementation
Row-Level Security	PostgreSQL RLS policies on every table enforce: WHERE tenant_id = current_setting('app.current_tenant'). Even raw SQL queries are filtered.
Branding	Each tenant has a settings JSONB column storing logo URL, primary color, and company name. The mobile app and web dashboard read these on login.
Data Isolation	No application-level filtering needed. The database itself prevents cross-tenant data access. This is the gold standard for SaaS security.

2. Page-by-Page Breakdown — Complete Application Map

This section documents EVERY page/screen in the techBuilder MVP, organized by user role. Each page includes its purpose, who uses it, key features, what makes it technically interesting, data interactions, and complexity assessment.

Total unique pages/screens in MVP: 58

Role	Pages	Page Names
□ Worker (Labourer)	6	Worker Home, Worker Photo Upload, Worker View Task, Worker Attendance History, Worker Profile, Worker Leave Request
□ Driver	8	Driver Home, Driver Start Day Log, Driver End Day Log, Driver Fuel Entry, Driver Expense Entry, Driver Vehicle Issue Report, Driver Daily Summary, Driver Trip Photo Upload
□ Worker-Head (Mistri)	8	Worker-Head Home, Worker-Head Crew Attendance, Worker-Head Upload on Behalf, Worker-Head Progress Report, Worker-Head Material Log, Worker-Head Issue Report, Worker-Head Daily Note, Worker-Head Expense Entry
□ Sub-Manager (Site Manager)	11	Sub-Manager Home / Site Dashboard, Sub-Manager Attendance View, Sub-Manager Daily Photo Upload, Sub-Manager Evening Report, Sub-Manager Task Assignment, Sub-Manager Expense Management, Sub-Manager Progress Tracker, Sub-Manager Worker List, Sub-Manager Vehicle View, Sub-Manager Issue Tracker, Sub-Manager Material Request
□ Manager (Company Owner/Admin)	14	Manager Dashboard, Manager All Sites View, Manager Attendance Overview, Manager Fleet Overview, Manager Activity Feed, Manager Expense Dashboard, Manager Approve Expenses, Manager Worker Directory, Manager Vehicle Directory, Manager Reports & Downloads, Manager Assign Workers, Manager Assign Vehicles, Manager Push Notifications Center, Manager Site Detail View
□ Platform (Super Admin)	5	Super Admin Dashboard, Tenant Management, Subscription & Billing, Platform Audit Log, System Backups
□ Shared / Cross-Role Pages	6	Login / Authentication, Company Creation / Onboarding, GPS Tracking Consent, Settings, Photo Gallery, Notification Center

2.1 □ Worker (Labourer) — 6 Pages

Worker Home

Purpose: Ultra-simple landing screen with 3-4 large buttons. This is the ONLY screen most workers will ever use.

Key Features:

- Clock IN button (big green)
- Clock OUT button (big red)

- Upload Photo button
- View Today's Task (if assigned)

What Makes This Page Special: Must work in Hindi. Must work offline. Must load in under 2 seconds on low-end Android phones (₹5,000-8,000 range). All buttons must be thumb-reachable in one hand.

Data Interactions: READ: today's attendance status, assigned task. WRITE: clock-in/out event with GPS coordinates.

Complexity Assessment: Offline-first: clock events queue locally and sync when network returns. GPS must be captured at exact moment of tap, not lazily.

Worker Photo Upload

Purpose: Capture and upload a work photo that automatically gets GPS coordinates, timestamp, and worker identity stamped on it.

Key Features:

- In-app camera (not gallery picker)
- Auto GPS + time overlay/watermark
- Optional text note
- Upload queue indicator

What Makes This Page Special: Photos must be compressed from ~4MB to ~300KB before upload. Watermark is burned into image server-side (not just metadata). Offline queue holds photos until connectivity returns.

Data Interactions: WRITE: photo blob to R2 storage, photo metadata (GPS, time, worker_id, project_id) to PostgreSQL.

Complexity Assessment: Image compression pipeline on device. Watermark compositing. Background upload queue with retry logic.

Worker View Task

Purpose: Read-only view of what this worker is supposed to do today. Assigned by Sub-Manager or Worker-Head.

Key Features:

- Task title + description
- Status badge (pending/in-progress/done)
- Mark as Complete button

What Makes This Page Special: Extremely simple UI. If no task is assigned, show 'No task for today' with a friendly illustration. Avoid overwhelming the worker with information.

Data Interactions: READ: task assigned to this worker for today. WRITE: task status update (mark complete).

Complexity Assessment: Low. Simple read/write. But must handle offline task completion and sync later.

Worker Attendance History

Purpose: Worker can see their own attendance record for the current month. Builds trust and transparency.

Key Features:

- Calendar view with present/absent/leave markers
- Total days worked this month
- Total hours (if tracked)

What Makes This Page Special: Read-only. Worker cannot edit their own attendance. This page exists to build trust — workers can verify they won't be cheated on pay day.

Data Interactions: READ: attendance records for current user, current month.

Complexity Assessment: Low. Simple calendar rendering with data overlay.

Worker Profile

Purpose: Basic profile page showing worker's identity, emergency contact, and role.

Key Features:

- Photo, name, phone number
- Role badge
- Emergency contact
- Company/site assignment

What Makes This Page Special: Serves as a digital ID card. Could be shown as a QR code for site entry verification in future.

Data Interactions: READ/WRITE: user profile fields.

Complexity Assessment: Low. Standard profile CRUD.

Worker Leave Request

Purpose: Simple form to request a day off. Replaces the current process of calling the manager.

Key Features:

- Date picker (single or range)
- Reason dropdown + optional text
- Status tracker (pending/approved/rejected)

What Makes This Page Special: Must be extremely simple. One screen, three fields, one button. Worker gets push notification when approved/rejected.

Data Interactions: WRITE: leave request. READ: past requests and their status.

Complexity Assessment: Low. Simple form + approval workflow.

2.2 □ Driver — 8 Pages

Driver Home

Purpose: Dashboard showing today's assigned vehicle, quick action buttons for start/end day logging, and fuel entry.

Key Features:

- Today's vehicle card (name, type, plate number)
- Start Day button
- End Day button
- Log Fuel button
- Today's summary (KM/hours so far)

What Makes This Page Special: Must show different UI based on vehicle type: Truck shows KM-focused interface, JCB/Crane shows Hours-focused interface. This is configured by admin per vehicle.

Data Interactions: READ: assigned vehicle, today's logs. WRITE: none (navigation hub).

Complexity Assessment: Medium. Vehicle type detection changes the entire UI flow.

Driver Start Day Log

Purpose: Driver captures the vehicle's starting state before first trip of the day.

Key Features:

- Camera to photograph odometer/hour meter
- Number input for KM or Hours reading
- GPS auto-capture of starting location
- Timestamp auto-capture

What Makes This Page Special: For KM-type vehicles: odometer photo + KM reading. For Hours-type vehicles: hour meter photo + hours reading. Photo is mandatory — cannot skip.

Data Interactions: WRITE: vehicle_log entry (type: start, photo_url, reading, gps, timestamp, driver_id, vehicle_id).

Complexity Assessment: Medium. Camera integration with forced capture. Dual UI path based on vehicle type.

Driver End Day Log

Purpose: Same as Start Day but captures ending state. System auto-calculates distance/hours for the day.

Key Features:

- Camera for ending odometer/hour meter photo
- Number input for ending reading
- Auto-calculated: Today's KM = end - start
- Auto-calculated: Today's Hours = end - start

What Makes This Page Special: Validates that end reading > start reading. Shows today's total prominently. If multiple drivers used the vehicle, each driver's contribution is calculated separately.

Data Interactions: WRITE: vehicle_log entry (type: end). READ: today's start log to calculate difference.

Complexity Assessment: Medium. Multi-driver calculation — if Driver A drove 50km in morning and Driver B drove 30km in evening, both are logged separately under the same vehicle for the same day.

Driver Fuel Entry

Purpose: Log a fuel fill-up with receipt photo as proof.

Key Features:

- Amount in ₹
- Liters filled
- Receipt photo (mandatory)
- GPS auto-capture (proves driver was at a fuel station)
- Auto-calculate: cost per liter

What Makes This Page Special: Receipt photo is mandatory to prevent fake fuel claims. GPS is auto-captured to cross-verify with known fuel station locations (future feature). Manager sees all fuel entries in their fleet overview.

Data Interactions: WRITE: fuel_entry (amount, liters, photo_url, gps, vehicle_id, driver_id, timestamp).

Complexity Assessment: Medium. Photo validation + GPS capture + financial data entry.

Driver Expense Entry

Purpose: Log non-fuel expenses like toll, parking, chai, or vehicle repair costs.

Key Features:

- Amount in ₹
- Category dropdown (Toll, Parking, Repair, Food, Other)
- Optional receipt photo
- Optional note

What Makes This Page Special: Simpler than fuel entry — receipt photo is optional here. But category selection helps manager understand spending patterns.

Data Interactions: WRITE: expense_entry with driver_id, vehicle_id, category.

Complexity Assessment: Low. Simple form.

Driver Vehicle Issue Report

Purpose: Report a mechanical problem or breakdown with photo evidence.

Key Features:

- Issue type dropdown (Flat tire, Engine, Brakes, Electrical, Other)
- Description text field
- Photo(s) of the issue
- Severity selector (Can continue / Needs attention / Breakdown)

What Makes This Page Special: Severity 'Breakdown' triggers an immediate push notification to the Manager. This is the construction equivalent of a production incident alert.

Data Interactions: WRITE: issue_report with vehicle_id, severity, photos. TRIGGER: push notification to manager if severity = breakdown.

Complexity Assessment: Medium. Real-time notification trigger + multi-photo upload.

Driver Daily Summary

Purpose: Auto-generated card showing today's complete driving activity.

Key Features:

- Total KM or Hours driven today
- Fuel consumed today
- Expenses logged today
- Trips completed
- Vehicle status

What Makes This Page Special: 100% auto-calculated from the day's log entries. Driver doesn't input anything here — it's a read-only summary that the driver and manager both see.

Data Interactions: READ: aggregation of today's vehicle_logs, fuel_entries, expense_entries for this driver.

Complexity Assessment: Medium. Requires aggregation queries across multiple tables.

Driver Trip Photo Upload

Purpose: Upload trip-related photos like loading proof, delivery confirmation, or material transport evidence.

Key Features:

- Camera with GPS + timestamp watermark
- Trip/purpose tag
- Note field
- Upload queue indicator

What Makes This Page Special: Same photo pipeline as Worker photo upload but tagged to a vehicle and trip rather than a project.

Data Interactions: WRITE: photo blob + metadata tagged to vehicle_id and trip context.

Complexity Assessment: Same as worker photo upload — image compression + watermark + offline queue.

2.3 Worker-Head (Mistri) — 8 Pages

Worker-Head Home

Purpose: Enhanced worker dashboard with additional crew management and upload-on-behalf capabilities.

Key Features:

- All Worker features (clock in/out, photo, task)
- Crew overview (who's present today)
- Upload on behalf of others
- Quick report buttons

What Makes This Page Special: This role exists because many construction workers don't own smartphones. The Mistri (experienced senior worker) carries a phone and handles digital operations for their crew of 5-15 workers.

Data Interactions: READ: crew list, their attendance, assigned tasks. WRITE: all worker operations + on-behalf operations.

Complexity Assessment: High. On-behalf operations require selecting which worker you're acting for, which adds a person-picker step to every action.

Worker-Head Crew Attendance

Purpose: Mark attendance for all workers in their crew. This is the primary way most workers get marked present.

Key Features:

- List of crew members with checkboxes
- Present / Absent / Half-day toggle per worker
- Bulk mark all present
- Submit button

What Makes This Page Special: This replaces the paper attendance register. The Mistri stands at the site entrance at 6-7 AM and checks off who showed up. Must work offline — sites often have no network in the morning.

Data Interactions: WRITE: attendance_record for each crew member (marked_by: worker_head_id).

Complexity Assessment: Medium. Batch write of multiple attendance records. Offline queue for the entire batch.

Worker-Head Upload on Behalf

Purpose: Upload photos, mark tasks, or log data on behalf of a specific worker who doesn't have a phone.

Key Features:

- Worker picker (select from crew)
- Action picker (Photo / Task Update / Other)

- The actual action form (same as worker would see)
- Attribution: 'Uploaded by [Mistri] on behalf of [Worker]'

What Makes This Page Special: Every action taken through this screen gets dual attribution — the data belongs to the worker, but the audit trail shows the Mistri uploaded it. This is critical for accountability.

Data Interactions: WRITE: same as worker actions, but with `uploaded_by` and `on_behalf_of` fields.

Complexity Assessment: High. Requires a selection layer before every action, and dual attribution in the data model.

Worker-Head Progress Report

Purpose: Upload progress photos with descriptive notes about what work was completed.

Key Features:

- Photo capture with GPS + watermark
- Work description text area
- Progress percentage slider
- Tag to project milestone

What Makes This Page Special: This is the Mistri's version of 'sending photos to the WhatsApp group in the evening.' Instead of WhatsApp, it goes directly into the project's photo gallery with proper metadata.

Data Interactions: WRITE: `progress_photo` with description, percentage, `milestone_id`.

Complexity Assessment: Medium. Same photo pipeline + structured metadata.

Worker-Head Material Log

Purpose: Record what construction materials were used today.

Key Features:

- Material picker (Cement, Steel, Sand, Bricks, etc.)
- Quantity input with unit
- Optional photo of material
- Daily running total

What Makes This Page Special: This is how the company tracks material consumption. Critical for detecting theft or wastage. The Mistri logs what was used; the manager compares against what was purchased.

Data Interactions: WRITE: `material_usage` (`material_type`, quantity, unit, date, `project_id`, `logged_by`).

Complexity Assessment: Low-Medium. Simple form + running total calculation.

Worker-Head Issue Report

Purpose: Report site problems like structural issues, water leakage, material shortage, or safety hazards.

Key Features:

- Issue type categorization
- Description + photos
- Severity level
- Escalation flag

What Makes This Page Special: Issues flagged here appear on the Sub-Manager's and Manager's dashboards. High-severity issues trigger push notifications.

Data Interactions: WRITE: issue (type, description, photos, severity, project_id). TRIGGER: notification if high severity.

Complexity Assessment: Medium. Multi-photo + notification trigger.

Worker-Head Daily Note

Purpose: End-of-day text summary of what happened. Replaces the WhatsApp evening message.

Key Features:

- Text area for free-form notes
- Timestamp
- Project tag
- Optional voice-to-text input

What Makes This Page Special: This is literally replacing 'Good evening sir, today we completed the foundation work for block B. 15 workers were present.' that currently goes on WhatsApp.

Data Interactions: WRITE: daily_note (text, project_id, date, author_id).

Complexity Assessment: Low. Text field + timestamp.

Worker-Head Expense Entry

Purpose: Log small expenses for materials bought locally (nails, small tools, etc.).

Key Features:

- Amount
- Category
- Receipt photo (optional)
- Description

What Makes This Page Special: Same as driver expense entry but tagged to a project instead of a vehicle.

Data Interactions: WRITE: expense with project_id, logged_by.

Complexity Assessment: Low.

2.4 Sub-Manager (Site Manager) — 11 Pages

Sub-Manager Home / Site Dashboard

Purpose: Overview of their assigned site — attendance, progress, expenses for today.

Key Features:

- Today's attendance count (present/absent/late)
- Today's expense total
- Recent activity feed
- Quick action buttons
- Pending approvals badge

What Makes This Page Special: This is the nerve center for one construction site. The sub-manager manages 20-100 workers across a single project. They need to see everything about their site at a glance.

Data Interactions: READ: attendance aggregation, expense sum, recent activities for their project.

Complexity Assessment: Medium. Multiple aggregation queries. Real-time update when workers clock in.

Sub-Manager Attendance View

Purpose: See and manage attendance for all workers on their site.

Key Features:

- List of all workers with today's status
- Approve/reject pending clock-ins
- Manual attendance override
- Attendance summary (count by status)

What Makes This Page Special: Two attendance modes: (1) Worker/Mistri marks attendance, sub-manager approves. (2) Sub-manager takes attendance directly. Configurable per company.

Data Interactions: READ: all attendance records for this site/today. WRITE: attendance approvals, manual entries.

Complexity Assessment: Medium. Approval workflow + real-time updates.

Sub-Manager Daily Photo Upload

Purpose: Upload morning and evening site photos as proof of daily progress.

Key Features:

- Morning photo section
- Evening photo section
- GPS + timestamp watermark
- Note per photo

What Makes This Page Special: This directly replaces sending photos to WhatsApp. Photos go into the project gallery and are visible to the Manager instantly.

Data Interactions: WRITE: site_photos with morning/evening tags.

Complexity Assessment: Same photo pipeline as others.

Sub-Manager Evening Report

Purpose: Write the daily summary message — what happened today, what's planned tomorrow.

Key Features:

- Text area for today's summary
- Worker count summary (auto-filled)
- Expense summary (auto-filled)
- Progress updates
- Issues flagged

What Makes This Page Special: Auto-fills data from the day's entries. Sub-manager just adds context and submits. This becomes the official daily record for the site.

Data Interactions: READ: day's aggregated data. WRITE: daily_report with structured + free-text content.

Complexity Assessment: Medium. Aggregation + template pre-filling.

Sub-Manager Task Assignment

Purpose: Assign daily tasks to workers and worker-heads.

Key Features:

- Task creation form (title, description, assignee)
- Assign to individual or crew
- Priority flag
- Track completion status

What Makes This Page Special: Tasks assigned here appear on the Worker's home screen. Simple task model — no dependencies, no subtasks. Just: assign, do, complete.

Data Interactions: WRITE: task (title, assignee_id, project_id, date). READ: task statuses.

Complexity Assessment: Low-Medium. Basic CRUD + push notification on assignment.

Sub-Manager Expense Management

Purpose: Log site expenses and approve expenses submitted by worker-heads.

Key Features:

- Add new expense (same form as worker-head)
- Pending approvals list
- Approve/reject with comment
- Daily/weekly expense summary

What Makes This Page Special: Multi-level approval: Worker-Head submits → Sub-Manager approves → Manager sees in their overview. Sub-manager is the first approval gate.

Data Interactions: READ: pending expenses, expense history. WRITE: new expenses, approval decisions.

Complexity Assessment: Medium. Approval workflow state machine.

Sub-Manager Progress Tracker

Purpose: Update work completion percentage per milestone or task.

Key Features:

- Milestone list with progress bars
- Update % slider
- Photo evidence for progress claim
- History of progress updates

What Makes This Page Special: Manager uses this to report to the company owner how much of the project is complete. Critical for billing milestones and client communication.

Data Interactions: READ/WRITE: milestone progress records.

Complexity Assessment: Low-Medium.

Sub-Manager Worker List

Purpose: View all workers assigned to their site with contact info and attendance summary.

Key Features:

- Worker list with photo, name, role
- Attendance rate (this month)
- Quick call/message action
- Filter by crew/role

What Makes This Page Special: Quick reference for the site manager to find any worker's info.

Data Interactions: READ: workers assigned to this project.

Complexity Assessment: Low. List view + filtering.

Sub-Manager Vehicle View

Purpose: See which vehicles are assigned to their site today.

Key Features:

- Vehicle list with driver, status
- Today's KM/hours
- Fuel consumption

What Makes This Page Special: Read-only view of fleet activity on their site. Cannot reassign vehicles (that's the Manager's job).

Data Interactions: READ: vehicles assigned to this project.

Complexity Assessment: Low.

Sub-Manager Issue Tracker

Purpose: View all reported issues for their site and escalate to Manager if needed.

Key Features:

- Issue list with severity badges
- Escalate to Manager button
- Add response/update
- Close resolved issues

What Makes This Page Special: Issues from Worker-Heads, Mistris, and their own observations. Unresolved high-severity issues escalate to Manager dashboard.

Data Interactions: READ: issues for this project. WRITE: status updates, escalations.

Complexity Assessment: Medium. State machine for issue lifecycle.

Sub-Manager Material Request

Purpose: Request materials from the main office/warehouse.

Key Features:

- Material type + quantity
- Urgency level
- Required by date
- Request status tracker

What Makes This Page Special: Request goes to Manager/Admin for approval. Tracks fulfillment status.

Data Interactions: WRITE: material_request. READ: request statuses.

Complexity Assessment: Low-Medium. Request + approval workflow.

2.5 Manager (Company Owner/Admin) — 14 Pages

Manager Dashboard

Purpose: Bird's eye view of the entire company — all sites, all workers, all vehicles, all expenses in one screen.

Key Features:

- KPI cards: total workers today, active vehicles, total expenses today, active sites
- Activity feed from all sites (scrollable)
- Pending approvals count
- Quick navigation to any site

What Makes This Page Special: This is the most data-dense page in the app. Must aggregate real-time data from potentially 5-20 sites, 100-500 workers, and 20-50 vehicles. Performance optimization is critical.

Data Interactions: READ: aggregations across ALL projects, workers, vehicles, expenses for this tenant.

Complexity Assessment: HIGH. Multi-table aggregation, potentially heavy query. Consider materialized views or pre-computed rollups. Real-time updates via WebSocket when new data arrives.

Manager All Sites View

Purpose: List/grid of all active construction sites with quick status indicators.

Key Features:

- Site cards with name, location, worker count, progress %
- Status indicators (on-track, delayed, issues)
- Quick navigation to site detail
- Search + filter

What Makes This Page Special: One construction company might manage 3-20 simultaneous projects. This page helps the owner prioritize which sites need attention.

Data Interactions: READ: project list with aggregated metrics.

Complexity Assessment: Medium. Per-project aggregation.

Manager Attendance Overview

Purpose: Cross-site attendance view — total workforce status today.

Key Features:

- Total present/absent/late across all sites
- Per-site breakdown
- Per-worker drill-down
- Attendance trend chart (week/month)

What Makes This Page Special: Answers the question: 'How many of my 200 workers showed up today across all sites?' Critical for payroll and resource planning.

Data Interactions: READ: attendance aggregation across all projects.

Complexity Assessment: Medium-High. Cross-project aggregation + charting.

Manager Fleet Overview

Purpose: See all vehicles — where they are, what they did today, fuel consumption.

Key Features:

- Vehicle list with today's KM/hours, fuel
- Driver assignment
- Vehicle status (active/idle/maintenance)
- Cost per vehicle summary

What Makes This Page Special: Critical for detecting fuel theft and vehicle misuse. The odometer photo + KM reading cross-verification is the key integrity check.

Data Interactions: READ: vehicle_logs, fuel_entries across all vehicles.

Complexity Assessment: Medium. Multi-vehicle aggregation.

Manager Activity Feed

Purpose: Chronological feed of all uploads, reports, photos, and events from all sites.

Key Features:

- Scrollable timeline
- Filter by site, type, role
- Photo previews inline
- Tap to expand details

What Makes This Page Special: This is the 'WhatsApp replacement' for the Manager. Instead of scrolling through 5 WhatsApp groups, they scroll through one unified, structured, searchable feed.

Data Interactions: READ: activity_log across all projects, ordered by time.

Complexity Assessment: Medium. Infinite scroll with filtering. Potentially high volume — pagination critical.

Manager Expense Dashboard

Purpose: Total spending across all sites — today, this week, this month.

Key Features:

- Total expense today (all sites)
- Category breakdown pie chart
- Per-site comparison bar chart
- Pending approval queue
- Expense trend line chart

What Makes This Page Special: Answers: 'Where is my money going?' The category breakdown (Material 40%, Labor 30%, Fuel 15%, Misc 15%) is the most-viewed chart.

Data Interactions: READ: expense aggregation across all projects. WRITE: approval decisions.

Complexity Assessment: High. Multi-dimensional aggregation + charting + approval workflow.

Manager Approve Expenses

Purpose: Review and approve/reject expenses submitted by sub-managers.

Key Features:

- Pending expense list with receipt photos
- Amount, category, submitter, site
- Approve / Reject with optional comment
- Batch approve

What Makes This Page Special: The final approval gate. Manager sees the receipt photo, the amount, and who submitted it. Batch approval speeds up the end-of-day review.

Data Interactions: READ: pending expenses. WRITE: approval status.

Complexity Assessment: Medium. Approval state machine + photo viewing.

Manager Worker Directory

Purpose: Complete directory of all workers across all sites.

Key Features:

- Search by name, phone, site, role
- Worker card with photo, attendance rate, contact info
- Reassign worker to different site
- Worker history (which sites they've worked on)

What Makes This Page Special: Centralized people database. Important for reassigning workers between sites based on need.

Data Interactions: READ: all workers for this tenant. WRITE: site reassignment.

Complexity Assessment: Medium. Search + filtering + reassignment logic.

Manager Vehicle Directory

Purpose: Complete vehicle registry with documents, maintenance, and assignment.

Key Features:

- Vehicle list with type, plate, driver, site
- Document storage (RC, insurance, PUC photos)
- Maintenance history
- Reassign to different site/driver

What Makes This Page Special: Centralized fleet management. Document expiry alerts are a key future feature.

Data Interactions: READ: all vehicles. WRITE: assignments, document uploads.

Complexity Assessment: Medium.

Manager Reports & Downloads

Purpose: Generate and download structured reports as PDF or Excel.

Key Features:

- Report type selector (Attendance, Expense, Vehicle, Daily Summary)
- Date range picker
- Site/project filter
- Download as PDF or Excel
- Schedule recurring reports (future)

What Makes This Page Special: Hindi PDF rendering is a known challenge — requires Puppeteer for correct Devanagari text. Excel uses ExcelJS with full cell styling.

Data Interactions: READ: report-specific aggregation queries. GENERATE: PDF via Puppeteer, Excel via ExcelJS.

Complexity Assessment: HIGH. Report generation is computationally expensive. Must run server-side. Consider background job queue (BullMQ) for large reports.

Manager Assign Workers

Purpose: Move workers between sites based on project needs.

Key Features:

- Source site picker
- Worker selection (multi-select)
- Target site picker
- Effective date

What Makes This Page Special: Construction projects have variable labor needs. A foundation phase might need 50 workers, but finishing needs only 10. Workers move between sites frequently.

Data Interactions: WRITE: worker-project assignment records.

Complexity Assessment: Medium. Must handle effective dating and prevent double-assignment.

Manager Assign Vehicles

Purpose: Assign vehicles to sites and drivers.

Key Features:

- Vehicle picker
- Target site + driver assignment

- Duration (temporary or permanent)
- View current assignments

What Makes This Page Special: Vehicles are expensive assets. Tracking which vehicle is on which site is important for utilization and cost allocation.

Data Interactions: WRITE: vehicle-project-driver assignment.

Complexity Assessment: Medium.

Manager Push Notifications Center

Purpose: View all notifications and alerts.

Key Features:

- Notification list with type badges
- Mark as read
- Tap to navigate to source
- Filter by type (issues, expenses, attendance, etc.)

What Makes This Page Special: Centralizes all alerts. High-priority items (breakdowns, incidents, budget overruns) are highlighted.

Data Interactions: READ: notifications for this user.

Complexity Assessment: Low-Medium. Standard notification system.

Manager Site Detail View

Purpose: Deep-dive into a specific site's data — all workers, vehicles, expenses, photos for that site.

Key Features:

- Site KPIs (workers today, expenses, progress)
- Worker list for this site
- Vehicle list for this site
- Photo gallery for this site
- Expense breakdown for this site

What Makes This Page Special: This is what you see when you tap a site from the All Sites view. Everything filtered to one project.

Data Interactions: READ: all data filtered by project_id.

Complexity Assessment: Medium. Multiple section queries for a single project.

2.6 □ Platform (Super Admin) — 5 Pages

Super Admin Dashboard

Purpose: Platform-level overview — total tenants, total users, revenue, system health.

Key Features:

- Total companies, users, MRR
- Growth charts
- System health indicators
- Recent signups

What Makes This Page Special: This is YOUR dashboard as the platform owner. Shows how the business is doing.

Data Interactions: READ: platform-wide aggregations across all tenants.

Complexity Assessment: High. Cross-tenant aggregation (carefully isolated from tenant data).

Tenant Management

Purpose: Create, suspend, or delete construction companies from the platform.

Key Features:

- Company list with plan, users, status
- Create new tenant
- Suspend/activate tenant
- View tenant usage metrics

What Makes This Page Special: Core SaaS admin function. Suspending a tenant instantly locks out all their users.

Data Interactions: READ/WRITE: tenant records.

Complexity Assessment: Medium. Tenant lifecycle management.

Subscription & Billing

Purpose: Manage plans and payment collection via Razorpay.

Key Features:

- Plan configuration (Free, Basic, Pro)
- Razorpay subscription integration
- Payment history per tenant
- Failed payment handling

What Makes This Page Special: Uses Razorpay Subscriptions API with UPI AutoPay. Plans should be simple: ₹999/month (Basic), ₹2,999/month (Pro).

Data Interactions: READ/WRITE: subscription records, Razorpay webhooks.

Complexity Assessment: High. Payment gateway integration, webhook handling, subscription lifecycle.

Platform Audit Log

Purpose: Immutable log of all administrative actions.

Key Features:

- Who did what, when
- Filter by action type, admin, date
- Export logs

What Makes This Page Special: Security requirement. Every tenant creation, suspension, plan change is logged.

Data Interactions: READ: audit_log entries.

Complexity Assessment: Low-Medium.

System Backups

Purpose: Monitor automated backup status.

Key Features:

- Last backup timestamp
- Backup size
- Restore button (with confirmation)
- Backup schedule configuration

What Makes This Page Special: Automated daily PostgreSQL backups. This page is mostly for monitoring.

Data Interactions: READ: backup metadata.

Complexity Assessment: Low. UI is simple; the backup system itself is a DevOps concern.

2.7 Shared / Cross-Role Pages — 6 Pages

Login / Authentication

Purpose: Entry point for all users. Phone OTP for workers, email+password for managers.

Key Features:

- Phone number + OTP flow (primary)
- Email + password flow (secondary)
- Company code / invite link / QR scan to join
- Language selector (Hindi / English)

What Makes This Page Special: After first login, the user sees company creation flow (if owner) or lands in their role-specific home. Auto-redirect based on role.

Data Interactions: WRITE: auth session. READ: user role for routing.

Complexity Assessment: High. Multi-method auth, OTP integration (MSG91), role-based routing, tenant resolution.

Company Creation / Onboarding

Purpose: First-time company owner creates their firm account.

Key Features:

- Company name, logo upload
- Owner profile setup
- First project creation
- Invite team members

What Makes This Page Special: This is the tenant provisioning flow. Creates the company record, sets up the owner as admin, and creates the first project. Should take under 3 minutes.

Data Interactions: WRITE: tenant record, first project, owner user record.

Complexity Assessment: High. Multi-step wizard with tenant provisioning, database setup, and initial data seeding.

GPS Tracking Consent

Purpose: Legal consent capture before any GPS tracking begins.

Key Features:

- Consent explanation in simple Hindi/English
- Accept / Decline buttons
- Consent record storage
- Disable tracking if declined

What Makes This Page Special: LEGAL REQUIREMENT. Must be captured before the app reads GPS. Consent record is stored with timestamp, device info, and user acknowledgment.

Data Interactions: WRITE: consent_record (user_id, timestamp, device_info, consent_given: boolean).

Complexity Assessment: Low. Simple form + legal text. But CRITICAL for compliance.

Settings

Purpose: User preferences and app configuration.

Key Features:

- Language toggle (Hindi/English)
- Notification preferences
- Dark mode (future)
- About / Help / Contact Support

What Makes This Page Special: Minimal for MVP. Language toggle is the most important setting for construction workers.

Data Interactions: READ/WRITE: user preferences.

Complexity Assessment: Low.

Photo Gallery

Purpose: Browse all photos for a project, organized by date.

Key Features:

- Grid/list toggle
- Filter by date, uploader, type
- Full-screen photo viewer
- Download original

What Makes This Page Special: Potentially large datasets — a busy site might have 50-100 photos per day. Pagination and lazy loading are essential. Thumbnail generation on upload.

Data Interactions: READ: photos for a project, paginated.

Complexity Assessment: Medium. Image optimization, thumbnailing, lazy loading, pagination.

Notification Center

Purpose: All push notifications in one place.

Key Features:

- Notification list
- Read/unread status
- Tap to navigate to source
- Clear all

What Makes This Page Special: Universal across all roles. Content varies by role.

Data Interactions: READ: notifications for current user.

Complexity Assessment: Low.

3. Data Flow Diagrams

This section details the four most critical data flows in techBuilder. Each flow represents a core operational pathway that must work flawlessly — including offline.

3.1 Photo Upload Flow (Offline → Sync)

This is the most complex data flow in the system because it involves binary data (images), offline queuing, compression, watermarking, and background uploads.

Step	What Happens
1. CAPTURE	User taps camera button → react-native-vision-camera opens → photo captured with GPS EXIF data via enableLocation prop
2. COMPRESS	react-native-compressor reduces 4-5MB photo to 200-400KB at quality 0.7. This happens instantly on-device.
3. WATERMARK	react-native-view-shot renders a React Native overlay (date, time, GPS coordinates, user name, project name) on top of the photo and captures the composite image.
4. LOCAL SAVE	Compressed + watermarked photo saved to device filesystem. Metadata (GPS, time, user_id, project_id, file_path) written to local SQLite upload queue.
5. NETWORK CHECK	expo-network checks connectivity. If online → proceed to upload. If offline → photo stays in queue.
6. BACKGROUND UPLOAD	expo-background-fetch periodically checks the queue. For each pending photo: upload to Cloudflare R2 via presigned URL → update SQLite record with remote URL → delete local file.
7. RETRY LOGIC	If upload fails: increment retry count, apply exponential backoff (5s, 15s, 45s, 2min, 5min). Max 10 retries. Limit 2-3 concurrent uploads to avoid overwhelming mobile networks.
8. SERVER PROCESSING	NestJS receives upload confirmation → writes photo metadata to PostgreSQL (with tenant_id via RLS) → triggers WebSocket event for real-time dashboard update.
9. THUMBNAIL	Server generates thumbnail (200px width) for gallery views. Stored alongside original in R2.
10. GALLERY UPDATE	Manager's dashboard and site gallery refresh to show the new photo. Sub-manager sees it in their activity feed.

3.2 Location Tracking Flow

GPS tracking is used for three purposes in techBuilder: (1) verifying clock-in/out location, (2) tracking vehicles, and (3) tagging photos. In the MVP, we focus on on-demand GPS capture (at moment of action) rather than continuous background tracking.

Use Case	How It Works	Technical Implementation	Battery Impact
Clock In/Out	One-time GPS capture at the moment of tap.	expo-location foreground permission. requestForegroundPermissionsAsync()	Low battery impact. Single

Use Case	How It Works	Technical Implementation	Battery Impact
	Stored as lat/lng in attendance record. Used to verify worker was at the job site.	→ <code>getCurrentPositionAsync()</code> . Accuracy: GPS (high) if available, fallback to network.	GPS read per event.
Photo Tagging	GPS captured at moment of photo via <code>VisionCamera's enableLocation</code> . Burned into watermark + stored in metadata.	<code>react-native-vision-camera</code> <code>enableLocation</code> prop reads from EXIF. Overlay rendered via <code>react-native-view-shot</code> .	Minimal battery impact. Piggybacks on camera session.
Vehicle Tracking	Driver manually logs start/end location. GPS captured at fuel fill-ups. NO continuous tracking in MVP.	Same one-time capture as clock-in. Future Phase 2: continuous background tracking with expo-location background task.	MVP avoids continuous tracking to reduce complexity and battery drain. Phase 2 adds it.
Live Location Share	On-demand: Manager requests 'where are you?' → Worker/Driver shares current location once.	Push notification triggers the worker's app to capture GPS and send back via API. Not continuous.	Simple request-response pattern. No persistent connection needed.

3.3 Report Generation Flow

Reports are generated on-demand by the Manager. They can be downloaded as PDF or Excel. This is a server-side process because: (1) aggregation queries are heavy, (2) Hindi PDF rendering requires Puppeteer, and (3) Excel styling requires ExcelJS.

Step	Description
1. USER REQUEST	Manager selects report type (Attendance, Expense, Vehicle, Daily Summary), date range, and site filter. Taps 'Generate'.
2. API CALL	Frontend sends POST <code>/api/reports/generate</code> with parameters. Returns a <code>job_id</code> immediately (async processing).
3. JOB QUEUE	NestJS pushes the report generation task to BullMQ (Redis-backed job queue). Frontend polls for completion status.
4. DATA QUERY	BullMQ worker executes the aggregation query against PostgreSQL. For example: attendance report = attendance records + worker names + site names for the date range.
5. PDF GENERATION	For PDF: Puppeteer renders an HTML template with the queried data. This handles Hindi text correctly. Output: PDF buffer.
6. EXCEL GENERATION	For Excel: ExcelJS creates a workbook with styled headers, data rows, and summary formulas. Supports Hindi in cell content.
7. FILE STORAGE	Generated file uploaded to Cloudflare R2 with a time-limited URL (expires in 24 hours).

Step	Description
8. NOTIFICATION	Job status updated to 'complete'. Frontend receives the download URL. Push notification sent if the user navigated away.
9. DOWNLOAD	User clicks download → browser fetches the file from R2 → file saved to device/desktop.

3.4 Offline Sync Flow (PowerSync)

Offline-first is not optional for construction SaaS — it's the most critical architectural decision. Indian construction sites frequently have zero network coverage, especially in early morning hours when attendance is taken.

Aspect	How It Works
Local Database	PowerSync maintains a local SQLite database on the device that mirrors a subset of the PostgreSQL data relevant to the current user. When the app opens, the user sees data instantly from SQLite — no loading spinner.
Write Path	All writes go to local SQLite FIRST. The app responds immediately (optimistic UI). PowerSync queues the write and syncs to PostgreSQL when online. This means clock-in, photo metadata, expense entries all work without internet.
Sync Rules	PowerSync Sync Rules define which data syncs to which device. Example: a Worker only gets their own attendance records and tasks for their assigned project. A Manager gets aggregated data for all their projects.
Conflict Resolution	Default: last-write-wins with server timestamps. For attendance records: if the same worker has two conflicting entries, the server-side timestamp wins. In practice, conflicts are rare because each user writes their own data.
Binary Files	Photos and receipt images are NOT synced through PowerSync. They use a separate upload queue (see Photo Upload Flow). Only the metadata (URL, GPS, timestamp) syncs through PowerSync.
Initial Sync	First app open downloads the full dataset for the user. Subsequent syncs are incremental (only changed rows). A Worker with 30 days of attendance records might sync 2-3KB initially.
Connectivity Detection	@react-native-community/netinfo monitors network state. When transitioning from offline to online, PowerSync automatically processes the queue. The app shows a sync indicator in the header.

4. Complexity & Risk Analysis

This section identifies the hardest parts of building techBuilder and the risks that could delay or derail development. This is the 'here be dragons' map.

4.1 Hardest Parts of the System

Challenge	Risk Level	Description	Mitigation
Offline Sync Architecture	CRITICAL	PowerSync + SQLite integration with conflict resolution, partial sync rules, and binary file queuing. This is architecturally the most complex part of the system. Budget 3-4 weeks for this alone.	Start with PowerSync's React Native quickstart. Build a proof-of-concept with attendance sync before tackling the full schema.
Multi-Tenant Data Isolation	HIGH	PostgreSQL RLS policies must be correct on EVERY table, EVERY query. A single missed policy = data leak between tenants. Testing requires spinning up multiple tenants and verifying isolation.	Write RLS policies as part of the migration. Create an automated test suite that creates 2 tenants and verifies zero cross-contamination.
Hindi PDF Generation	HIGH	Devanagari ligatures break in jsPDF and @react-pdf/renderer. The only reliable solution is Puppeteer (headless Chromium). This adds a heavy dependency to the server.	Use Puppeteer from day one for all PDFs. Don't attempt client-side PDF generation for Hindi content.
Camera + GPS + Watermark Pipeline	MEDIUM-HIGH	Three libraries must work together: VisionCamera (capture), compressor (resize), view-shot (watermark). Getting the pipeline right with offline queuing takes iteration.	Build and test the photo pipeline as a standalone module before integrating into the app.
Role-Based UI Routing	MEDIUM	6 different roles, each with a different home screen and different navigation options. The same app binary must render completely different UIs based on who's logged in.	Use Expo Router with route groups: (worker)/, (driver)/, (manager)/, etc. Middleware handles redirect based on role.
Real-Time Dashboard Updates	MEDIUM	The Manager dashboard must update live when workers clock in, expenses are submitted, or issues are reported. WebSocket connection management + reconnection logic.	Use Socket.io with NestJS Gateway. Implement room-based broadcasting (one room per tenant).

4.2 Scaling Challenges

Scale	What Breaks	Solution
1-10 tenants	No scaling concerns. Single PostgreSQL instance handles everything. Neon free tier sufficient.	None needed.
10-100 tenants	Database queries slow down as data grows. Photo storage approaches R2 free tier limits.	Add database indexes. Move to Neon Launch (\$19/month). Implement query caching with Redis.
100-500 tenants	Real-time WebSocket connections reach thousands. Report generation becomes slow.	Add Redis for WebSocket pub/sub. Implement BullMQ for background jobs. Consider read replicas.
500-1000 tenants	Single database becomes a bottleneck. Need to consider sharding or dedicated databases for large tenants.	Evaluate database-per-tenant for top 10% of customers. Kubernetes for horizontal API scaling.

4.3 Security Concerns

Risk	Severity	Description	Mitigation
Cross-Tenant Data Leak	CRITICAL	RLS misconfiguration exposes one company's data to another. This is a company-ending bug.	Automated cross-tenant isolation tests in CI. Manual security audit before launch.
GPS Spoofing	MEDIUM	Workers could use GPS spoofing apps to fake their clock-in location.	Phase 2: detect mock locations via expo-location's isMocked field. Flag suspicious entries.
Photo Manipulation	MEDIUM	Workers could upload pre-existing photos instead of real-time captures.	Enforce in-app camera only (no gallery picker). GPS + timestamp watermark makes it harder to fake.
Financial Data Exposure	HIGH	Expense data, salary information, and P&L must be strictly role-gated.	API-level authorization checks. Never rely on frontend-only access control.
OTP Interception	LOW	Man-in-the-middle on SMS-based OTP.	Use WhatsApp OTP as primary (end-to-end encrypted). SMS as fallback only.

5. Feature Phases — Development Roadmap

Features are organized into three phases. Phase 1 (MVP) is what you build to launch and get your first 5-10 paying customers. Phase 2 adds depth. Phase 3 adds differentiation.

5.1 Phase 1 — MVP (Months 1-5)

Goal: Replace diary + WhatsApp for one construction company. Get 5-10 pilot customers.

Area	MVP Features
Platform Core	Company signup, Phone OTP login, Email login, Invite link to join, Role assignment (6 roles), Data isolation (RLS), Multi-project support, Offline-first mobile, Hindi + English, Push notifications
Worker	Home screen (3 buttons), Clock IN/OUT with GPS, Photo upload with watermark, View assigned task, Basic profile
Driver	Home screen (vehicle-based), Start/End day odometer photo + reading, Vehicle type config (KM vs Hours), Multi-driver per vehicle, Fuel entry with receipt photo, Daily summary card
Worker-Head	All Worker features, Crew attendance marking, Upload on behalf of others, Progress photo with notes, Daily text summary, Material usage log
Sub-Manager	Site dashboard, Take/approve attendance, Daily photo upload, Evening summary report, Task assignment, Expense logging + approval, Progress tracker
Manager	Company dashboard with KPIs, All sites overview, Cross-site attendance, Fleet overview, Activity feed, Expense dashboard + approvals, Worker + vehicle directories, PDF/Excel report downloads, Assign workers + vehicles
Super Admin	Platform dashboard, Tenant management, Razorpay subscription billing, Daily backups
Shared	GPS consent capture, Photo gallery, Notification center, Settings (language toggle)

5.2 Phase 2 — Growth (Months 6-10)

Goal: Add analytical depth, fleet management, and communication features. Target 25-50 paying customers.

Area	Phase 2 Features
Analytics	Weekly/monthly expense graphs, Income vs expenditure chart, Attendance trend charts, Per-vehicle cost analysis
Fleet Depth	Live GPS vehicle tracking, Route history replay, Geofence alerts, Maintenance reminders, Document expiry alerts, Fuel efficiency analytics
Communication	Site activity feed, Direct messaging (1:1), Incident report with photos, SOS emergency button
Expense Depth	OCR receipt scanning, Budget vs actual tracking, Petty cash management, Income tracking

Area	Phase 2 Features
Subcontractors	Invite sub firm, Limited permissions, Daily log submission
Platform	Feature flags per plan, Usage analytics, QR code onboarding, Company branding, Biometric login

5.3 Phase 3 — Pro (Months 11-18)

Goal: Enterprise features and AI differentiation. Target 100+ customers.

Area	Phase 3 Features
AI Features	AI daily report summary, Expense anomaly detection, Cost overrun prediction, Weather-based work planning
Advanced Reports	Custom report builder, 6-month/yearly trend analysis, Scheduled auto-reports, Custom graphs
Communication	Group chat per site, Voice notes, WhatsApp Business API, Company announcements
Project Mgmt	Gantt chart scheduling, Client portal, Bid estimation, Punch list management
Platform	Custom subdomain, Custom domain, Custom roles, Public API, Plugin marketplace

6. Initial Budget — What's Free, What's Not

techBuilder can launch at near-zero infrastructure cost by leveraging free tiers aggressively. Here's the exact breakdown for the first 6 months before any revenue.

6.1 Free Tier Stack (₹0/month)

Service	Purpose	Free Tier Limits	Notes
Vercel Hobby	Next.js hosting	100GB bandwidth, serverless functions	Cannot use for commercial projects. Apply for Startup Program (\$30K credits).
Neon Free Tier	PostgreSQL database	0.5GB storage, 5GB egress, scale-to-zero	Auto-pauses after 5 min idle. Sufficient for development + early testing.
Cloudflare R2 Free	Photo/file storage	10GB storage, 1M writes, 10M reads/month	Zero egress fees. Enough for first 5-10 pilot companies.
Expo Free	Mobile builds	15 builds/month, 1K MAU updates	Sufficient for development. May need \$19/month Starter for faster builds.
Firebase FCM	Push notifications	Unlimited messages, completely free	No limits on FCM for Android. The best free tier in all of SaaS.
GitHub Free	Code hosting + CI	Unlimited repos, 2000 CI min/month	Private repos included. GitHub Actions minutes are generous.
PostHog Free	Analytics + errors	1M events, 5K replays, 100K errors/month	Replaces Mixpanel + Sentry + FullStory. Incredible free tier.
Figma Starter	UI design	Unlimited editors, 3 collab files	Adequate for MVP design work.
Hoppscotch	API testing	Unlimited collections, fully free	Open-source Postman alternative.
Google Play Console	App publishing	One-time ₹2,100 (\$25) fee	Not monthly — one-time registration.

6.2 First Paid Services (When Revenue Starts)

Service	Cost	What You Get	When Needed
MSG91 SMS/OTP	₹500-1,000/month	~2,000-4,000 OTP messages. Prepaid wallet, no monthly commitment.	First real cost. Start when you have 50+ users logging in.
Vercel Pro	₹1,700/month (\$20)	Required for commercial use. Apply for Startup Program first.	Can delay by using Cloudflare Pages (\$5/month) initially.
Neon Launch	₹1,600/month (\$19)	10GB storage, auto-scaling. Move when free tier storage fills.	Usually needed around 20-30 active companies.

Service	Cost	What You Get	When Needed
Expo Starter	₹1,600/month (\$19)	High-priority builds, 5K MAU updates. Move when builds are slow.	Optional. Free tier works but builds are queued.
PowerSync Cloud	₹4,100/month (\$49)	1,000 peak connections. The offline sync engine.	Can self-host PowerSync open edition for free instead.
Domain Name	₹800/year (\$10)	techbuilder.app or similar.	One-time annual cost.

Total Month 1-3 (Development): ₹2,100 one-time (Google Play) + ₹0/month infrastructure

Total Month 4-6 (First Pilots): ₹3,000-5,000/month (MSG91 + Vercel Pro or Cloudflare Pages)

Total Month 7-12 (Growth): ₹8,000-12,000/month (add Neon Launch + Expo Starter)

7. Developer Skill Requirements

This section lists the exact skills needed for each discipline. Use this for hiring, self-assessment, or training planning.

Discipline	Level	Skills
Frontend Web	Must Have	Next.js 16 (App Router, Server Components, Server Actions), React 19, Tailwind CSS v4, shadcn/ui, TanStack Query v5, Zustand, React Hook Form + Zod, TypeScript
Frontend Web	Should Have	Recharts/Chart.js for graphs, TanStack Table for data grids, next-intl for Hindi/English, Puppeteer for PDF generation, ExcelJS for Excel export
Mobile	Must Have	React Native with Expo SDK 55, Expo Router, NativeWind (Tailwind for RN), react-native-vision-camera, expo-location, PowerSync + SQLite, react-native-compressor
Mobile	Should Have	react-native-view-shot for watermarks, react-native-mmkv for fast storage, @rnmapbox/maps for fleet tracking, expo-background-fetch, Expo Notifications + FCM setup
Backend	Must Have	NestJS v11 with Fastify, Drizzle ORM, PostgreSQL with RLS, JWT authentication, REST API design, File upload handling (multipart + presigned URLs)
Backend	Should Have	Socket.io for real-time, BullMQ for job queues, MSG91 API for OTP, Razorpay API for subscriptions, PostGIS for geospatial queries
DevOps	Must Have	Git + GitHub, GitHub Actions CI/CD, Vercel deployment, Neon PostgreSQL management, Cloudflare R2 setup, Docker basics
DevOps	Should Have	Turborepo monorepo management, EAS Build + Update, Database migration strategies, Log management, Monitoring setup (PostHog + Sentry)
System Design	Must Have	Multi-tenant SaaS patterns, Offline-first architecture, Role-based access control, API security (JWT, rate limiting), Database schema design
System Design	Should Have	Event-driven architecture, CQRS for read-heavy dashboards, Background job processing, WebSocket connection management, Data consistency in distributed systems

7.1 Minimum Team for MVP

Role	Responsibility	Duration
Lead Full-Stack Developer (You)	Own the architecture, build core backend + frontend. 60% of the codebase.	Months 1-5 continuous
Mobile Developer	Build the Expo app, camera pipeline, offline sync, GPS integration.	Months 2-5 (can be you if solo)
Part-time Designer	Figma designs for all 6 role-specific UIs. Mobile-first, Hindi-friendly.	Months 1-2 (40 hours total)
Part-time QA/Tester	Test on low-end Android phones at actual construction sites.	Months 4-5 (before launch)

8. Suggestions & Missing Features

8.1 Features Commonly Found in Competitors That You Should Add

Feature	Why It Matters	Suggested Phase
Weather Integration	Auto-fill weather conditions in daily reports. Construction work is weather-dependent. Use OpenWeather free tier (1,000 calls/day).	Phase 2
Document Storage	Store blueprints, contracts, permits per project. Simple file upload + folder structure. NOT document management — just organized storage.	Phase 1 (simple version)
Worker Wage Calculator	Auto-calculate wages from attendance: Days worked × Daily rate = Amount owed. This is what construction owners care about MOST on payday.	Phase 1 — HIGH PRIORITY
WhatsApp Sharing	One-tap share daily report summary to WhatsApp. Don't fight WhatsApp — integrate with it. Workers will still use WhatsApp; let them share from your app.	Phase 1
Site Map Pin	Drop a pin on Google Maps when creating a project. Used for geofence radius and location-based attendance verification.	Phase 1 (simple map pin)
Overtime Rules	Configure overtime rules per company (after 8 hours = 1.5x, after 10 = 2x). Auto-calculate in wage reports.	Phase 2
Material Purchase Orders	Create PO → send to vendor → track delivery → receive with photo. Completes the material lifecycle.	Phase 2
Client Progress Portal	Read-only link that the construction company shares with their client. Client sees progress photos, milestone status, and completion %. No login required — magic link access.	Phase 2 — DIFFERENTIATOR

8.2 Competitive Differentiators

These features don't exist in any competitor at the SMB price point:

- White-label multi-tenancy: No competitor offers this. Each company sees techBuilder as 'their' app.
- Offline-first architecture: Connectteam (biggest workforce management tool) has NO offline mode. This alone wins construction clients.
- Vehicle KM vs Hours dual tracking: No construction SaaS distinguishes between truck KM and JCB hours. Your drivers will love this.
- Multi-driver per vehicle: The concept that Vehicle X had 3 different drivers today, each logging their own KM — no competitor handles this.

- Odometer photo verification: The 'snap the meter' approach creates tamper-proof distance records. Unique to techBuilder.
- Worker-Head upload-on-behalf: Solving for the reality that most construction workers don't have smartphones. No competitor addresses this.
- ₹999-2,999/month pricing: Procore costs \$10,000+/year. Buildertrend costs \$5,000+/year. Your price point serves the 90% of construction firms that can't afford enterprise tools.

8.3 Critical Missing Items to Add Before Launch

Item	Details	When
Terms of Service + Privacy Policy	Legal requirement. Must be shown during signup. Cover GPS tracking, data storage, multi-tenant data handling. Consult a lawyer familiar with Indian IT Act.	BEFORE LAUNCH
Data Backup Strategy	Automated PostgreSQL backups. Neon provides point-in-time recovery. Set up a secondary backup to R2 weekly.	BEFORE LAUNCH
Error Monitoring	PostHog error tracking + Sentry for crash reports. Must be set up from Day 1 — you can't debug construction site issues without crash data.	DAY 1
User Feedback Channel	In-app 'Report a Problem' button. Links to a simple form or WhatsApp business number. Construction workers will have issues — make it easy to report.	BEFORE LAUNCH
App Store Optimization	Play Store listing: Hindi + English description, screenshots for each role, keywords targeting 'construction management app India'.	BEFORE LAUNCH
Onboarding Tutorial	2-3 screen walkthrough on first login for each role. Workers need to understand: Green button = arrive, Red button = leave. That's it.	BEFORE LAUNCH

9. Week-by-Week Sprint Plan — MVP (20 Weeks)

This is a realistic 20-week (5-month) sprint plan for building the techBuilder MVP with a team of 1-2 developers. Each sprint is 1 week.

Sprint	Focus	Key Deliverables
Week 1	Project Setup	Turborepo monorepo init, Next.js + Expo + NestJS apps created, Drizzle + PostgreSQL + Neon setup, GitHub Actions CI pipeline, Tailwind v4 + shadcn/ui configured
Week 2	Database Schema	Design and implement full PostgreSQL schema: tenants, users, projects, attendance, vehicles, vehicle_logs, fuel_entries, expenses, photos, tasks, notifications. RLS policies on ALL tables.
Week 3	Auth + Tenant	MSG91 OTP integration, email+password login, JWT issuance, company creation flow, invite link generation, role assignment. GPS consent screen.
Week 4	Worker Mobile App	Worker home screen, clock IN/OUT with GPS, basic profile page. PowerSync setup + SQLite local database. Offline clock-in queue.
Week 5	Camera Pipeline	VisionCamera integration, react-native-compressor setup, react-native-view-shot watermark overlay, offline photo queue, Cloudflare R2 upload with presigned URLs.
Week 6	Driver Mobile App	Driver home screen, start/end day log (odometer photo + KM/hours), vehicle type config, fuel entry form, multi-driver logic.
Week 7	Worker-Head Features	Crew attendance marking, upload on behalf, progress photo with notes, material usage log, daily text summary.
Week 8	Sub-Manager Part 1	Site dashboard, attendance view + approval, daily photo upload, evening summary report form.
Week 9	Sub-Manager Part 2	Task assignment, expense logging + approval workflow, progress tracker, worker list view, issue reporting.
Week 10	Manager Dashboard	Company-wide KPI dashboard, all sites overview, cross-site attendance view, fleet overview. Focus on aggregation queries + performance.
Week 11	Manager Features	Activity feed (infinite scroll), expense dashboard + approval, worker directory, vehicle directory.
Week 12	Manager Reports	PDF generation with Puppeteer (Hindi support), Excel generation with ExcelJS. Attendance, expense, vehicle, and daily summary reports.
Week 13	Real-Time	Socket.io integration in NestJS. Live attendance updates on dashboard. Push notification setup (Expo + FCM). Alert triggers for issues.
Week 14	Super Admin	Platform dashboard, tenant management, Razorpay subscription integration (UPI AutoPay), plan creation + assignment.
Week 15	Hindi + i18n	next-intl setup for web, expo-localization for mobile. Translate ALL worker-facing screens to Hindi. Manager screens can stay English-first.
Week 16	Offline Testing	Comprehensive offline testing: airplane mode testing for every write operation. PowerSync sync rule optimization. Edge case handling (sync failures, partial uploads).

Sprint	Focus	Key Deliverables
Week 17	UI Polish	Responsive design review (mobile + tablet + desktop). Loading states, error states, empty states. Skeleton screens. Performance optimization.
Week 18	QA + Field Testing	Install on actual construction workers' phones. Test at real construction sites. Fix discovered issues. Low-end Android device testing (₹5,000-8,000 phones).
Week 19	Security + Hardening	Penetration testing. Cross-tenant isolation verification. Rate limiting. Input validation. SQL injection prevention audit. API security review.
Week 20	Launch Prep	Play Store submission, production deployment, monitoring setup (PostHog + Sentry), backup verification, first customer onboarding support.

10. Project Structure — Monorepo Layout

techBuilder uses a Turborepo monorepo to share TypeScript types, validation schemas, and configuration across web, mobile, and backend.

Path	Type	Description
apps/web/	Next.js 16 dashboard	Admin and Manager web interface. Server-rendered pages, API routes for report generation, middleware for tenant resolution.
apps/mobile/	Expo SDK 55 app	Worker, Driver, Sub-Manager mobile app. Offline-first with PowerSync. Camera, GPS, push notifications.
apps/api/	NestJS v11 backend	REST API + WebSocket server. Business logic, auth, file processing, Razorpay webhooks.
packages/shared/	Shared TypeScript	Zod validation schemas, TypeScript types/interfaces, constants (role definitions, expense categories), utility functions. Used by ALL three apps.
packages/db/	Database layer	Drizzle ORM schema definitions, migration files, seed data, RLS policy definitions. Imported by apps/api/.
packages/ui/	Shared UI components	React components used by both web and mobile (where applicable). Primarily form components and display utilities.
packages/config-eslint/	ESLint config	Shared linting rules across all apps.
packages/config-ts/	TypeScript config	Base tsconfig.json extended by all apps.

11. Conclusion & Next Steps

techBuilder is a well-scoped, architecturally sound multi-tenant construction SaaS targeting the massively underserved Indian SMB construction market. The MVP includes 54+ unique pages across 6 user roles, built on a modern TypeScript stack that costs ₹0-5,000/month to operate.

Immediate Next Steps:

1. Set up the Turborepo monorepo with Next.js + Expo + NestJS
2. Design the PostgreSQL schema with RLS policies (Week 2 deliverable)
3. Build the auth + tenant creation flow (Week 3 deliverable)
4. Validate the camera pipeline as a standalone proof-of-concept
5. Set up PowerSync with a simple attendance sync prototype
6. Create Figma mockups for the Worker and Driver home screens

— End of Document —